

A First Look at Your Project's Size

CS 5388 / CS 4390

Team 1

Project: DamnParking

Step 1. List what Release 1 must do (as a team)

Start from the “in scope” items in your charter and the strongest needs from your interviews. Write 5 to 8 capabilities as what the system must do, not how. Then mark the hidden work each one will need using the key below.

Hidden work key: T = Testing S = Security or privacy I = Integration with other systems D = Data (collection, cleaning, migration) H = Hosting and deployment U = Usability and accessibility O = Documentation or training

#	Capability (what, not how)	Evidence (charter or interview)	Hidden work
Ex.	A student can buy a semester parking permit online.	Interview 4, Parking Services	T, S, I, H
1	Finding Available Parking	LotLens Interview [1/16], [2/16]	T, I, H,
2	Viewing live availability in parking lots	LotLens Interview [1/16], [2/16], [6/16], [7/16]	S, I, D,H
3	Find parking that will become available	LotLens Interview [1/16], [2/16], [7/16]	S, D, H, U
4	Input you current parking spot	LotLens Interview [1/16], [2/16], [7/16]	T, S, I, D
5	See what parking lots are within our observed system	Lot Lens Interview [7/16]	I, D, H, U
6	Parking map of a city	Lot Lens Interview [3/16], [6/16]	I, H, U
7	What type of parking is available (One-way parking, parallel parking, tight spaces, etc.)	Lot Lens Interview [4/16], [5/16], [6/16]	S, I, D, H, U

The example row is from the fictional Campus Parking System used in lecture.

Step 2. Take the outside view: your own experience *(as a team)*

As a team, pick the most similar thing any of you has built before: a class project, internship feature, hackathon app, or side project.

Question	Answer
What was it?	Athletyx MMA Hackathon Project
How long was it expected to take?	3 weeks
How long did it take?	2 nights and lots of redbull
What took longer than expected?	Getting the CNN model to be >75% accurate

Step 3. Take the outside view: similar products *(as a team, web search)*

Divide the searching, then decide together. Find three real products that solve a problem close to yours. Include at least one small product (a startup or an app from a small team) and at least one big product (a large company platform or a government system). For each, find out how long it took to reach its first public release, what went well, and what went wrong.

How to find the timeline and the story

1. Launch announcements, company blogs, and founder interviews often say when building started and when the product first went public.
2. Press coverage, app store version histories, and the Internet Archive's Wayback Machine help confirm first release dates.
3. Postmortems, conference talks, and, for government systems, GAO and Inspector General reports explain what worked and what did not.
4. Measure from the start of development to the first public release, and write down what that first release included. A limited first version and a full launch are different numbers.
5. Every date and figure needs a source you can link. If you cannot find one, write "unknown." AI tools may suggest products to look at, but they are not a source.

Example: a completed entry for a parking product

ParkEasy is fictional. It shows the level of detail to aim for, not real data.

Field	Entry
Product and company	ParkEasy, a four-person startup
Why it is similar	Drivers find and pay for parking from a phone, like our Campus Parking System.
Development started	January 2019 (founder blog post)
First public release	October 2019, in one city only (local news launch story)
Time to first release	About 9 months
What the first release included	A map of open lots and pay-by-phone. No reservations and no monthly permits.
Team size	Three engineers and one designer
What worked	Launching in one city first. Partnering with a single lot operator to get reliable availability data.
What did not work	Their own parking sensors gave bad availability data, so they switched to the operator's feed, adding about 3 months. Payment integration took twice as long as planned.
Sources	Founder blog post (link); local news launch story (link)

What this example tells a team building a parking app: plan for data and payment integration to take the longest, and scope the first release to one location.

Step 3 worksheet: similar products

	Product A (small)	Product B (big)	Product C (your choice)
Product and company	DAMNLINES.COM	ParkNav	BestParking
Why it is similar to your project	It follows the same philosophy of live footage and insight based on the live cctv. (wait times compared to open spots)	Uses real-time data and AI to help drivers find available parking, similar to our goal of helping students find available campus parking	Both help users find parking more easily. BestParking shows nearby parking locations and information, our app focuses on helping students find available campus parking spots.
Development started	March 2026	January 2015	2005
First public release	April 2026	January 2016	2007 as NYCgarages.com
Time to first release	1 Month	12 Months	Approximately 1-2 years, but the exact development timeline is unknown,
What the first release included	Live IP-camera feeds for 4 West Village restaurants, computer-vision-based people counting, estimated wait times, information about the best times to visit, and the ability for users to suggest/vote on future locations.	Real-time on-street parking availability predictions using machine learning and parking data.	Search for parking by address/location, view nearby garages on a map, and compare parking rates and specials.
Team size, if known	Solo project	14 to 17 employees	Team size is unknown
What worked	The MVP solved a very simple, recognizable problem: "Should I go there right now?" It combined existing cameras, computer vision, and queue mathematics instead of requiring complicated new hardware.	Hardware-light/data-driven approach, machine-learning parking predictions, and ability to scale across cities. ParkNav says its system can substantially reduce the need for physical parking sensors.	Simple parking search, centralized parking information, price comparison, and maps. The product later expanded to many cities and gained hundreds of thousand of users.
What did not work	Scaling and sustainability were major problems.	Accuracy and coverage depend on available data. Parking conditions change constantly,	Collecting accurate parking data was difficult and initially required manual data

	Product A (small)	Product B (big)	Product C (your choice)
		requiring multiple real-time and historical data sources. Current app updates also mention accuracy, stability, UI, and performance improvements	collection. Some parking operators also resisted having prices published.
Sources (links)	DamnLines About page; DamnLines Privacy Policy; DamnLines API documentation; Brooklyn Magazine, Apr. 14, 2026; New York Post, Apr. 20, 2026; Product Hunt; DamnLines App Store listing.	https://parknav.com/ https://inrix.com/press-releases/on-street-germany/	https://www.entrepreneur.com/starting-a-business/web-startup-will-find-the-best-parking-spot/225477 https://techcrunch.com/2013/02/28/best-parking-redesign-api

Step 4. Compare *(as a team)*

1. Look at the hidden-work column in Step 1. Which letter appears most often? Did any of your similar products struggle with that same kind of work?

The letters that appear most are Integration with other systems (I) and Hosting and Deployment (H). Our similar products struggled with the same work, especially with scaling, sustainability, and expanding the system to new locations.

2. How long did your similar products take to reach a first release, and with how many people? What does that suggest about a first release of your project?

Project DAMNLINES took approximately 1 month with 1 individual, while ParkNav took about 12 months with around 14-17 employees. This suggests that our first release should stay small and focus on the most important parking features, rather than trying to build the entire system at once.

3. Name one practice from the “what worked” rows your team will copy, and one mistake from the “what did not work” rows you will avoid.

One great thing we could copy from project DAMNLINES is their approach; it's really interesting, and they created a simple MVP using existing cameras rather than requiring new hardware. One thing we will seek to avoid is trying to scale to too many locations too quickly, as this will likely cause scaling and sustainability problems.

Our comparables suggest a first release would take 2 to 4 months with a team of 4.

Keep this sheet. You will use Step 1 to write user stories, and Steps 2 and 3 give you comparators for analogous estimation in Part 2.